

Laboratory
0x04

Expected delivery of lab_04.zip must include:

- **program_1.s**, **program_1_a.s**, and **program_1_b.s**
- This file, filled with information and possibly compiled in a **PDF** format.

Delivery date:

November 7th 2025

This lab will explore some of the concepts seen during the lessons, such as hazards, rescheduling, and loop unrolling. The first thing to do is to configure the GEM5 simulator with the ***Initial Configuration*** provided below:

```
INTEGER_ALU_LATENCY = 1
INTEGER_MUL_LATENCY = 1
INTEGER_DIV_LATENCY = 1
FLOAT_ALU_LATENCY = 4
FLOAT_MUL_LATENCY = 8
FLOAT_DIV_LATENCY = 20
```

- 1) Write an assembly program (**program_1.s**) for the RISC-V architecture able to compute the output (y) of a **neural computation** (see the Fig. below):

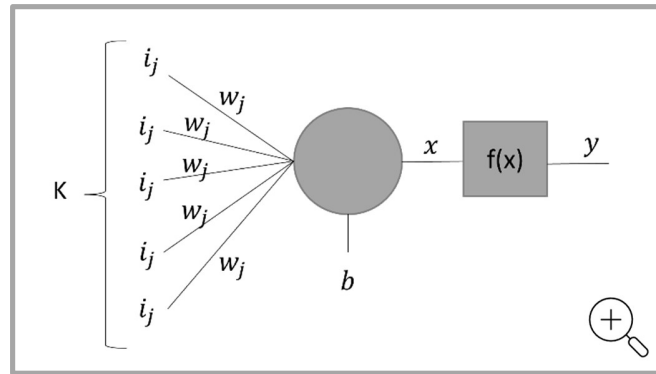
$$x = \sum_{j=0}^{K-1} i_j * w_j + b$$
$$y = f(x)$$

where, to prevent the propagation of NaN (Not a Number), the activation function f is defined as:

$$f(x) = \begin{cases} 0, & \text{if the exponent part of } x \text{ is equal to } 0x7FF \\ x, & \text{otherwise} \end{cases}$$

Assume the vectors i and w respectively store the inputs entering the neuron and the weights of the connections. They contain $K=16$ single precision **floating point** elements. Assume that b is a single precision **floating point** constant and is equal to $0xab$, and y is a single precision **floating point** value stored in memory.

Compute y .



Given the *Base Configuration*, run your program and extract the following information.

	Number of clock cycles	Total Instructions	CPI (Clock per Instructions)
program_1.s	330	151	2.18543

- Optimize the program by re-scheduling instructions to eliminate as many hazards as possible. Manually calculate the number of clock cycles for the new program (**program_1_a.s**) to execute and compare the results with those obtained by the simulator.
- Unroll the program (**program_1_a.s**) two times; If necessary, re-schedule instructions and increase the number of registers used. Manually calculate the number of clock cycles to execute the new program (**program_1_b.s**) and compare the results obtained with those obtained by the simulator.

Complete the following table with the obtained results:

Program	program_1.s	program_1_a.s	program_1_b.s
Clock cycles by hand	<u>310</u>	<u>246</u>	<u>180</u>
Clock cycles by simulation	<u>330</u>	<u>281</u>	<u>182</u>

Collect the Cycles Per Instruction (CPI) from the simulator for different programs

	program_1.s	program_1_a.s	program_1_b.s
CPI	<u>2.18543</u>	<u>1.86093</u>	<u>1.4918</u>

Compare the results obtained in 1) and provide some explanation if the results are different.

Eventual explanation:

Hand calculations assumed ideal conditions. The simulator accounts for real hardware constraints: structural hazards, memory conflicts, and variable latencies.

program_1_b.s is most accurate because fewer iterations reduce cumulative error and behavior is more predictable.

CPI improve from 2.18 to 1.49: loop unrolling reduces stalls by 32% through fewer branches and independent accumulators.

- c. Try different unrolls for the program (**program_1_a.s**); If necessary, re-schedule instructions and increase the number of registers used. Manually calculate the number of clock cycles to execute the new program (**program_1_b.s**) and compare the results obtained with those obtained by the simulator.

Complete the following table with the obtained results:

Program	Number of Loop Unrolling	Code size [bytes]	Clock Cycles [CC]
program_1_a.s	-	<u>140</u>	<u>281</u>
	2	<u>168</u>	<u>182</u>
	4	<u>200</u>	<u>146</u>
	8	<u>284</u>	<u>148</u>
	16	<u>388</u>	<u>134</u>

In order to calculate the code size of your program you can open in *./programs/you_program/your_program.dump* file to retrieve the compiled disassembled code of your executable in which you have the effective addresses from which the code will be executed.

For example (on the right), for *your program* you have your disassembled file on 32 bit addresses.

The screenshot shows a disassembler window for 'program1.elf'. It displays assembly instructions with their corresponding addresses. A red arrow points to the address '00010004' labeled 'Start Address'. Another red arrow points to the address '00000073' labeled 'End Address'. The instructions include various operations like 'auipc', 'addi', 'flw', 'lw', 'fmul.s', and 'ecall'.

```

1  program1.elf:  file format elf32-littleliscv
2
3
4  Disassembly of section .text:
5
6  00010004 <start>:
7      00001097      auipc ra,0x1
8      00008093      addi ra,ra,100 # 110f8 <__DATA_BEGIN__>
9      0000a487      flw  fs1,0(ra)
10     00001097      auipc ra,0x1
11     00008093      addi ra,ra,104 # 11108 <v2>
12     0000a487      flw  fs1,0(ra)
13     00001097      auipc ra,0x1
14     00008093      addi ra,ra,108 # 11118 <v3>
15     0000a183      lw   sp,0(ra)
16     00001097      auipc ra,0x1
17     00008093      addi ra,ra,112 # 11128 <v0>
18     0000a183      lw   sp,0(ra)
19     00001097      auipc ra,0x1
20     00008093      addi ra,ra,52 # 110f8 <__DATA_BEGIN__>
21     00001117      auipc sp,0x1
22     00001117      addi sp,sp,60 # 11108 <v2>
23     0000a087      flw  ft1,0(ra)
24     00012187      flw  ft2,0(sp)
25     0000a187      flw  ft3,0(ra)
26     00012207      flw  ft4,0(sp)
27
28  000100e4 <Main>:
29     1020f2d3      fmul.s ft5,ft1,ft2
30     1020f2d3      fmul.s ft5,ft1,ft2
31
32  000100ec <End>:
33     00000513      li   a7,0
34     00000000      li   a7,0
35     00000073      ecall
36
37

```